

Qualité de code en Python

Bonnes pratiques, tests, sécurité et DevSecOps

Document pédagogique complet

7 avril 2026

Table des matières

1	Introduction	4
1.1	Objectif du document	4
1.2	Criticité	4
1.3	Contenu du document	4
1.4	Approche pédagogique	4
2	Commentaires trompeurs et code peu expressif	5
2.1	Mauvais code	5
2.2	Bon code	5
2.3	Explication	5
3	Nommage métier et lisibilité	6
3.1	Mauvais code	6
3.2	Bon code	6
3.3	Explication	6
4	Nombres magiques et duplication	7
4.1	Mauvais code	7
4.2	Bon code	7
4.3	Explication	7
5	Dictionnaires fragiles vs dataclasses	8
5.1	Mauvais code	8
5.2	Bon code	8
5.3	Explication	8
6	Docstrings et style conforme	9
6.1	Mauvais code	9
6.2	Bon code	9
6.3	Explication	9
7	Gestion incorrecte de None	10
7.1	Mauvais code	10
7.2	Bon code	10
7.3	Explication	10
8	Traitement explicite de None	11
8.1	Mauvais code	11
8.2	Bon code	11
8.3	Explication	11

9 Exceptions avalées	12
9.1 Mauvais code	12
9.2 Bon code	12
9.3 Explication	12
10 Valeurs de retour implicites	13
10.1 Mauvais code	13
10.2 Bon code	13
10.3 Explication	13
11 Concaténation de chaînes en boucle	14
11.1 Mauvais code	14
11.2 Bon code	14
11.3 Explication	14
12 Complexité excessive	15
12.1 Mauvais code	15
12.2 Bon code	15
12.3 Explication	15
13 Gestion des sous-processus	16
13.1 Mauvais code	16
13.2 Bon code	16
13.3 Explication	16
14 Tests unitaires absents	17
14.1 Mauvais code	17
14.2 Bon code	17
14.3 Explication	17
15 Analyse statique avec Pylint	18
15.1 Mauvais code	18
15.2 Bon code	18
15.3 Explication	18
16 Vérification des types avec MyPy	19
16.1 Mauvais code	19
16.2 Bon code	19
16.3 Explication	19
17 Analyse de sécurité avec Bandit	20
17.1 Mauvais code	20
17.2 Bon code	20
17.3 Explication	20
18 Sécurité des dépendances	21
18.1 Mauvais code	21
18.2 Bon code	21
18.3 Explication	21

19 Complexité avec Radon	22
19.1 Mauvais code	22
19.2 Bon code	22
19.3 Explication	22
20 Couverture de tests	23
20.1 Mauvais code	23
20.2 Bon code	23
20.3 Explication	23
21 Inspection continue avec SonarQube	24
21.1 Mauvais code	24
21.2 Bon code	24
21.3 Explication	24
22 Conclusion	25

Chapitre 1

Introduction

1.1 Objectif du document

Ce document a pour objectif de présenter les bonnes pratiques essentielles en langage Python à travers une approche comparative entre **mauvais** et **bon code**. Il vise à améliorer la qualité, la robustesse et la maintenabilité des programmes en mettant en évidence les erreurs courantes et leurs corrections.

1.2 Criticité

Python permet de développer rapidement, mais cette simplicité apparente peut masquer des défauts importants. Une mauvaise pratique peut rapidement conduire à :

- du code difficile à relire et à maintenir
- des erreurs d'exécution liées à des contrats flous
- des problèmes de sécurité
- une complexité croissante difficile à maîtriser

Adopter des règles strictes permet de structurer le code, d'améliorer sa fiabilité et de faciliter son industrialisation.

1.3 Contenu du document

Ce document couvre les aspects fondamentaux de la qualité en Python :

- Respect des conventions de style, de nommage et de documentation
- Réduction de la complexité et suppression des duplications
- Gestion explicite des erreurs, des valeurs absentes et des contrats
- Écriture de code lisible, testable et maintenable
- Mise en place des tests unitaires et mesure de couverture

1.4 Approche pédagogique

Chaque chapitre suit une structure simple et efficace :

- un exemple de **mauvais code**
- une version corrigée (**bon code**)
- une **explication** claire du problème et de la solution

Cette approche permet de comprendre rapidement les erreurs fréquentes et d'adopter les bons réflexes en situation réelle.

Chapitre 2

Commentaires trompeurs et code peu expressif

2.1 Mauvais code

```
error_value = False # Reset error value to true
```

2.2 Bon code

```
error_value = False
```

2.3 Explication

Un commentaire faux ou inutile rend le code plus dangereux qu'utile. Quand le nommage est clair, le commentaire devient souvent inutile. Un commentaire doit expliquer l'intention ou la contrainte, pas répéter le code.

Chapitre 3

Nommage métier et lisibilité

3.1 Mauvais code

```
intv = 200  
time.sleep(intv / 1000)
```

3.2 Bon code

```
stabilisation_delay_ms = 200  
time.sleep(stabilisation_delay_ms / 1000)
```

3.3 Explication

Un nom trop court ou ambigu ralentit la lecture. Un nom métier explicite permet de comprendre le rôle de la variable sans effort.

Chapitre 4

Nombres magiques et duplication

4.1 Mauvais code

```
if month == 3:
    day_of_month += 59
elif month == 4:
    day_of_month += 90
elif month == 5:
    day_of_month += 31 + 28 + 31 + 30
```

4.2 Bon code

```
MONTH_LENGTHS = [31, 28, 31, 30, 31, 30,
                  31, 31, 30, 31, 30, 31]

def get_day_of_year(month: int, day_of_month: int) -> int:
    return sum(MONTH_LENGTHS[:month - 1]) + day_of_month
```

4.3 Explication

Les nombres magiques et les branches répétitives rendent le code fragile et difficile à relire. Il vaut mieux centraliser les données dans une structure claire et éviter la duplication.

Chapitre 5

Dictionnaires fragiles vs dataclasses

5.1 Mauvais code

```
user = {"nom": "Alice", "role": "admin"}  
print(user["nmo"])
```

5.2 Bon code

```
from dataclasses import dataclass  
  
@dataclass(frozen=True)  
class User:  
    nom: str  
    role: str  
  
user = User(nom="Alice", role="admin")
```

5.3 Explication

Un dictionnaire libre ne fournit aucun contrat fort sur les champs attendus. Une **dataclass** apporte une structure explicite, une meilleure lisibilité et une meilleure compatibilité avec les outils d'analyse.

Chapitre 6

Docstrings et style conforme

6.1 Mauvais code

```
import sys

def Calculer_Somme(A, B):
    result = A + B
    return result
```

6.2 Bon code

```
"""Fonctions de calcul mathematique basiques."""

def calculer_somme(valeur_a: float, valeur_b: float) -> float:
    """
    Calcule et retourne la somme de deux nombres.

    Args:
        valeur_a: Premier operande.
        valeur_b: Second operande.

    Returns:
        Somme des deux valeurs.
    """
    resultat = valeur_a + valeur_b
    return resultat
```

6.3 Explication

Un code Python professionnel doit respecter les conventions de nommage, d'indentation et de documentation. Les docstrings améliorent la compréhension, la génération de documentation et les analyses automatiques.

Chapitre 7

Gestion incorrecte de None

7.1 Mauvais code

```
def find_user(user_id):  
    if user_id not in db:  
        return None  
    return db[user_id]
```

7.2 Bon code

```
from typing import Optional  
  
def find_user(user_id: str) -> Optional["User"]:  
    if user_id not in db:  
        return None  
    return db[user_id]
```

7.3 Explication

Retourner `None` peut être acceptable, mais il faut le rendre explicite dans le contrat de la fonction. Le typage permet d'indiquer clairement à l'appelant qu'une absence de valeur est possible.

Chapitre 8

Traitement explicite de None

8.1 Mauvais code

```
def get_user_name(user_id):  
    return find_user(user_id).name
```

8.2 Bon code

```
def get_user_name(user_id: str) -> str:  
    user = find_user(user_id)  
    if user is None:  
        raise ValueError("Utilisateur introuvable")  
    return user.name
```

8.3 Explication

Enchaîner directement sur un résultat potentiellement nul produit des erreurs d'exécution. Il faut traiter explicitement le cas d'absence.

Chapitre 9

Exceptions avalées

9.1 Mauvais code

```
try:
    save()
except Exception:
    pass
```

9.2 Bon code

```
import logging

logger = logging.getLogger(__name__)

try:
    save()
except OSError as exc:
    logger.exception("Erreur lors de la sauvegarde")
    raise RuntimeError("Impossible de sauvegarder") from exc
```

9.3 Explication

Un `except` vide masque complètement l'erreur et rend le diagnostic difficile. Il faut au minimum journaliser, puis propager ou transformer l'exception si nécessaire.

Chapitre 10

Valeurs de retour implicites

10.1 Mauvais code

```
def compute(value):  
    if value < 0:  
        return  
    return value * 2
```

10.2 Bon code

```
from typing import Optional  
  
def compute(value: int) -> Optional[int]:  
    if value < 0:  
        return None  
    return value * 2
```

10.3 Explication

Une fonction qui retourne parfois une valeur et parfois rien devient ambiguë. Il faut rendre ce contrat explicite avec un type adapté.

Chapitre 11

Concaténation de chaînes en boucle

11.1 Mauvais code

```
resultat = ""  
for i in range(10000):  
    resultat += str(i)
```

11.2 Bon code

```
parties = []  
for i in range(10000):  
    parties.append(str(i))  
  
resultat = "".join(parties)
```

11.3 Explication

Concaténer une chaîne dans une boucle crée de nombreux objets intermédiaires. La méthode `join` est plus efficace et plus idiomatique.

Chapitre 12

Complexité excessive

12.1 Mauvais code

```
def evaluer_statut(age, abonne, premium):  
    if age > 18:  
        if abonne:  
            if premium:  
                return "VIP"  
            else:  
                return "Membre"  
        else:  
            return "Visiteur"  
    else:  
        return "Mineur"
```

12.2 Bon code

```
def evaluer_statut(age, abonne, premium):  
    if age <= 18:  
        return "Mineur"  
    if not abonne:  
        return "Visiteur"  
    if premium:  
        return "VIP"  
    return "Membre"
```

12.3 Explication

Des conditions imbriquées en cascade augmentent la complexité cognitive. Les retours anticipés rendent souvent le code plus lisible et plus simple à tester.

Chapitre 13

Gestion des sous-processus

13.1 Mauvais code

```
import subprocess

cmd = input("Commande: ")
subprocess.call(cmd, shell=True)
```

13.2 Bon code

```
import subprocess

allowed = ["status", "version"]
cmd = input("Commande: ")

if cmd not in allowed:
    raise ValueError("Commande interdite")

subprocess.run(["tool", cmd], check=True)
```

13.3 Explication

L'usage de `shell=True` avec une entrée utilisateur ouvre la porte à des injections de commande. Il faut restreindre les commandes autorisées et passer les arguments sous forme de liste.

Chapitre 14

Tests unitaires absents

14.1 Mauvais code

```
def add(a, b):  
    return a + b
```

14.2 Bon code

```
def add(a: int, b: int) -> int:  
    return a + b  
  
def test_add() -> None:  
    assert add(2, 2) == 4
```

14.3 Explication

Sans test, une régression peut passer inaperçue. Même un test très simple augmente fortement la confiance dans le comportement attendu.

Chapitre 15

Analyse statique avec Pylint

15.1 Mauvais code

```
import sys

def Calculer_Somme(A, B):
    result = A + B
    return result
```

15.2 Bon code

```
pip install pylint
pylint mon_script.py --output-format=parseable > pylint-report.txt
```

15.3 Explication

Pylint détecte les problèmes de style, de nommage, de structure et de robustesse. Il permet d'industrialiser la qualité de code dès les premières phases du développement.

Chapitre 16

Vérification des types avec MyPy

16.1 Mauvais code

```
def increment(x):  
    return x + 1  
  
increment("abc")
```

16.2 Bon code

```
def increment(x: int) -> int:  
    return x + 1
```

16.3 Explication

Le typage statique optionnel aide à détecter des incohérences avant l'exécution. MyPy complète utilement les tests et l'analyse de style.

```
pip install mypy  
mypy mon_script.py --strict
```

Chapitre 17

Analyse de sécurité avec Bandit

17.1 Mauvais code

```
import pickle

data = pickle.loads(user_input)
```

17.2 Bon code

```
import json

data = json.loads(user_input)
```

17.3 Explication

Certaines bibliothèques ou pratiques introduisent des risques de sécurité importants. Bandit aide à repérer automatiquement ce type de vulnérabilités dans le code Python.

```
pip install bandit
bandit -r . -f xml -o bandit-report.xml
```

Chapitre 18

Sécurité des dépendances

18.1 Mauvais code

```
# requirements.txt non verifie  
flask==0.10  
requests==2.4.0
```

18.2 Bon code

```
pip install safety  
safety check -r requirements.txt
```

18.3 Explication

Une part importante des vulnérabilités provient des dépendances tierces. Il faut scanner régulièrement les bibliothèques utilisées.

Chapitre 19

Complexité avec Radon

19.1 Mauvais code

```
def process(a, b, c, d, e):  
    if a:  
        if b:  
            if c:  
                if d:  
                    if e:  
                        return 1  
    return 0
```

19.2 Bon code

```
def process(a, b, c, d, e):  
    if not a:  
        return 0  
    if not b:  
        return 0  
    if not c:  
        return 0  
    if not d:  
        return 0  
    if not e:  
        return 0  
    return 1
```

19.3 Explication

Une complexité trop élevée rend le code difficile à maintenir et à tester. Radon permet de mesurer la complexité cyclomatique.

```
pip install radon  
radon cc mon_script.py -a
```

Chapitre 20

Couverture de tests

20.1 Mauvais code

```
# Code de production jamais execute en test
```

20.2 Bon code

```
pip install pytest coverage
coverage run -m pytest --junitxml=pytest-report.xml test_mon_script.py
coverage xml
```

20.3 Explication

La couverture mesure la part du code réellement exécutée par les tests. Elle permet d'identifier des zones non vérifiées ou du code mort.

Chapitre 21

Inspection continue avec SonarQube

21.1 Mauvais code

```
# Aucun quality gate centralise
```

21.2 Bon code

```
sonar.projectKey=mon_projet_python
sonar.projectName=Projet Python DevSecOps
sonar.sources=src
sonar.tests=tests
sonar.python.version=3.11

sonar.python.pylint.reportPaths=pylint-report.txt
sonar.python.bandit.reportPaths=bandit-report.xml
sonar.python.coverage.reportPaths=coverage.xml
sonar.python.xunit.reportPath=pytest-report.xml
```

21.3 Explication

SonarQube centralise les rapports de qualité, de sécurité, de couverture et de dette technique. Cela permet de mettre en place une vraie quality gate dans la CI.

Chapitre 22

Conclusion

Un code Python de qualité professionnelle en approche DevSecOps repose sur :

- un code lisible, bien nommé et documenté ;
- une gestion explicite des erreurs, des valeurs absentes et des contrats ;
- des fonctions simples, peu complexes et peu dupliquées ;
- des tests unitaires et une couverture mesurée ;
- une analyse statique de style, de type et de sécurité ;
- une intégration continue avec quality gate.

Ces pratiques permettent de produire un code plus robuste, plus maintenable, plus sûr et mieux industrialisé.